AGENTIC AI · SKILL-BASED EVALUATION

EvidenceWeave

Turning raw computer logs into a verified story of a cyber attack

TEAM MEMBERS

1. Nithya Maria Mathew
URK23CS7004

2. Demian Sharwin Xaxa
URK23CS7011

THE BASICS

A cyber attack leaves traces

Every computer keeps a diary of what happens on it. That diary is called a **log**. After an attack, investigators must read the logs to find out what the attacker did.

3

kinds of logs

Login log, web-server log, Windows log

3,907

lines to read

In just one of our hard test cases

6

attack stages hidden inside

Buried among normal, harmless activity

THE BASICS

What do logs look like?

Each source writes in its own format and has its own clock, so they are hard to read together.

Login log

Who tried to log in, who used admin rights, what jobs ran

```
Failed password for invalid user admin from 203.0.113.45
```

Web-server log

Who asked the website for which page, and what the answer was

```
203.0.113.45 asked for /.env and got 404
```

Windows log

Logons, programs started, scheduled tasks on a Windows computer

```
Event 4624: network logon succeeded for user deploy
```

THE BASICS

Why not just ask an AI?

AI writes fluent summaries, but it can also **make things up**. This is called a hallucination. For example it may:

- ⚠ Change one digit of an IP address
- ⚠ Shift a time by a few minutes
- ⚠ Claim an action no log ever shows

OUR QUESTION

How can a small, free, local AI be allowed to speak only when every sentence is backed by evidence?

OUR IDEA

Three jobs, three different workers

1 · RULES

Find the attack

Simple, fixed checks such as “8 failed logins in 10 minutes”. The same input always gives the same answer.

2 · AI

Explains it in plain English

A small local model only rewrites facts the rules already proved. It does not decide what happened.

3 · CHECKER

Proves every sentence

Each sentence is checked against the log lines it cites. If it fails it is fixed or removed.

THE ATTACK WE LOOK FOR

A six-stage break-in

Stage 1

Brute force

Guessing passwords again and again

Stage 2

Initial access

One guess works and the attacker gets in

Stage 3

Privilege escalation

Gets admin power on the machine

Stage 4

Persistence

Leaves a back door to return later

Stage 5

Lateral movement

Jumps to a second computer on the network

Stage 6

Web reconnaissance

Scans the website for secret files

THE TEST DATA

Fake attacks with a hidden answer key

A generator writes realistic logs and plants the attack. It also writes an answer key that the analysis code can never read, so we can score the system honestly. No real personal data is used.

Easy case

787

log lines, little noise

Hard case

3,907

log lines, with three traps

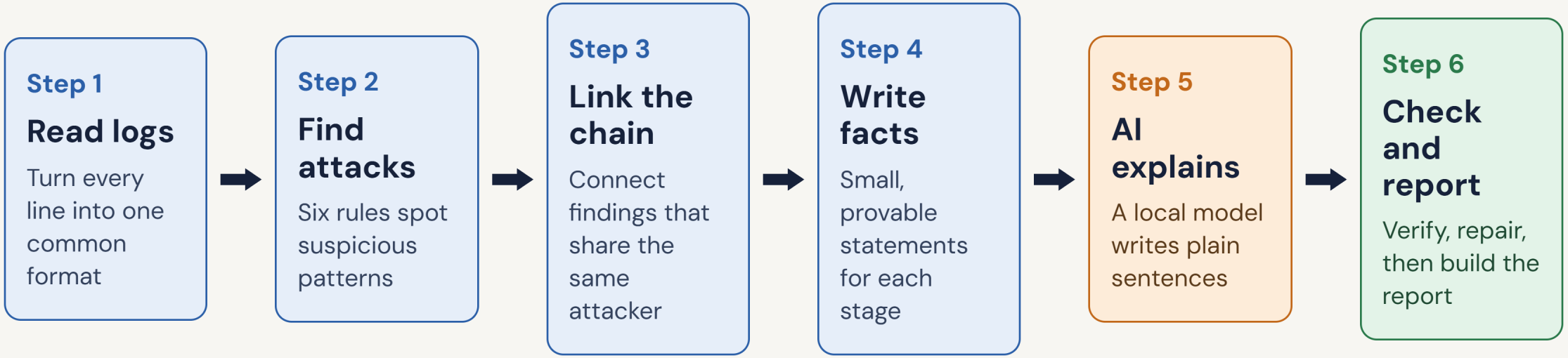
Decoy attacker
Attacks but never gets in

Clock skew
Windows clock runs 90 seconds fast

Deleted evidence
One log line was removed

THE SYSTEM

How it works in six steps



THE SYSTEM

The full architecture

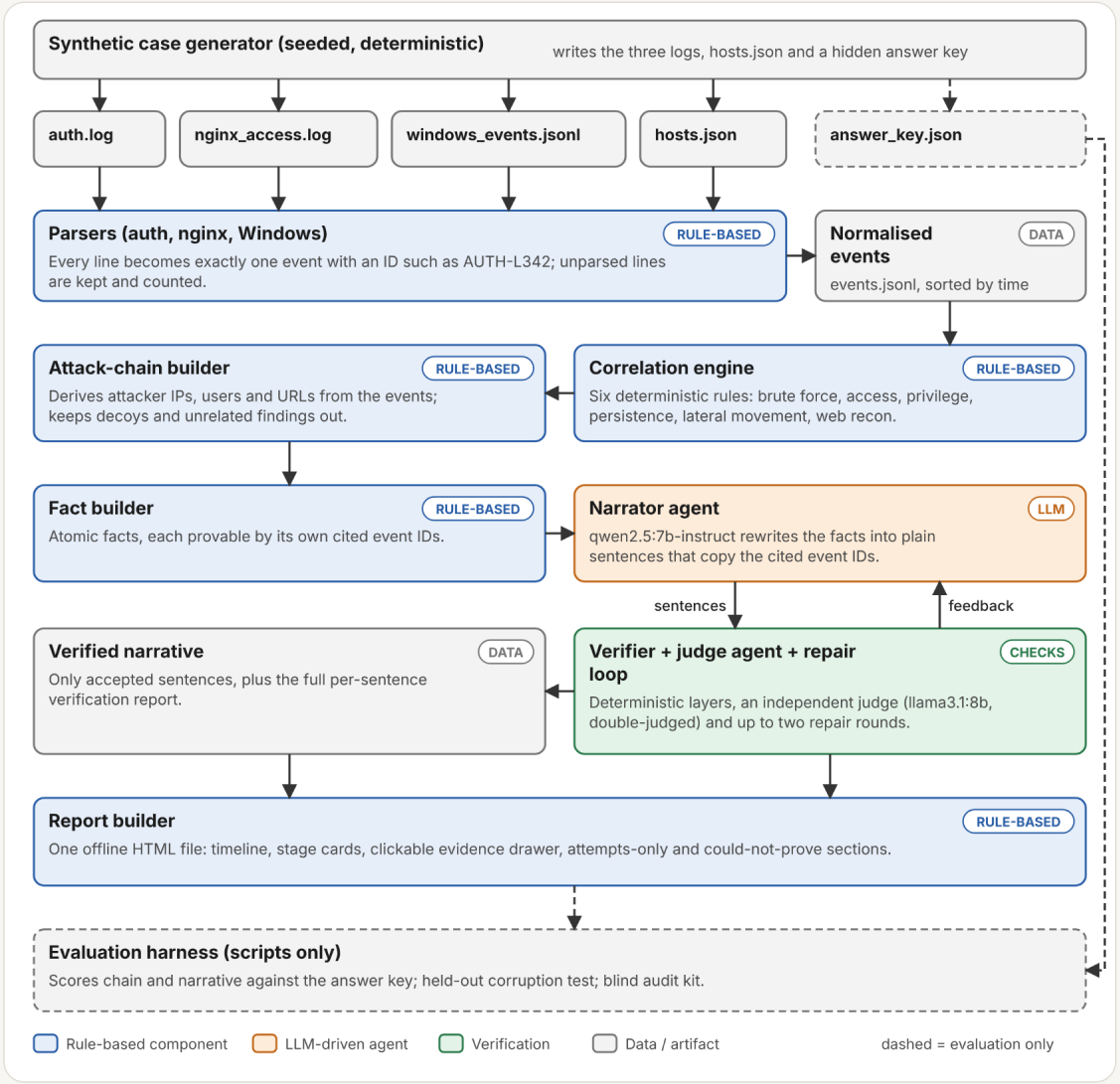
Raw logs go in at the top. A verified report comes out at the bottom.

Blue boxes: rule-based code

Orange boxes: the AI agents

Green box: verification

Dashed path: the answer key is read only by the scoring scripts, never by the pipeline



STEP 1 · READ LOGS

Every line becomes one event

RAW LINE 342 OF THE LOGIN LOG

```
Oct 27 02:10:07 web01 sshd: Failed
password for invalid user admin
from 203.0.113.45
```



ONE COMMON EVENT

event_id: AUTH-L342

host: web01 · **user:** admin

src_ip: 203.0.113.45

action: ssh_login_failed

Traceable
AUTH-L342 means line 342 of the login log

Nothing dropped
A line we cannot parse is kept and counted

Clock-aware
Cross-machine links allow a 5-minute gap

STEP 2 · FIND ATTACKS

Six simple rules, no AI

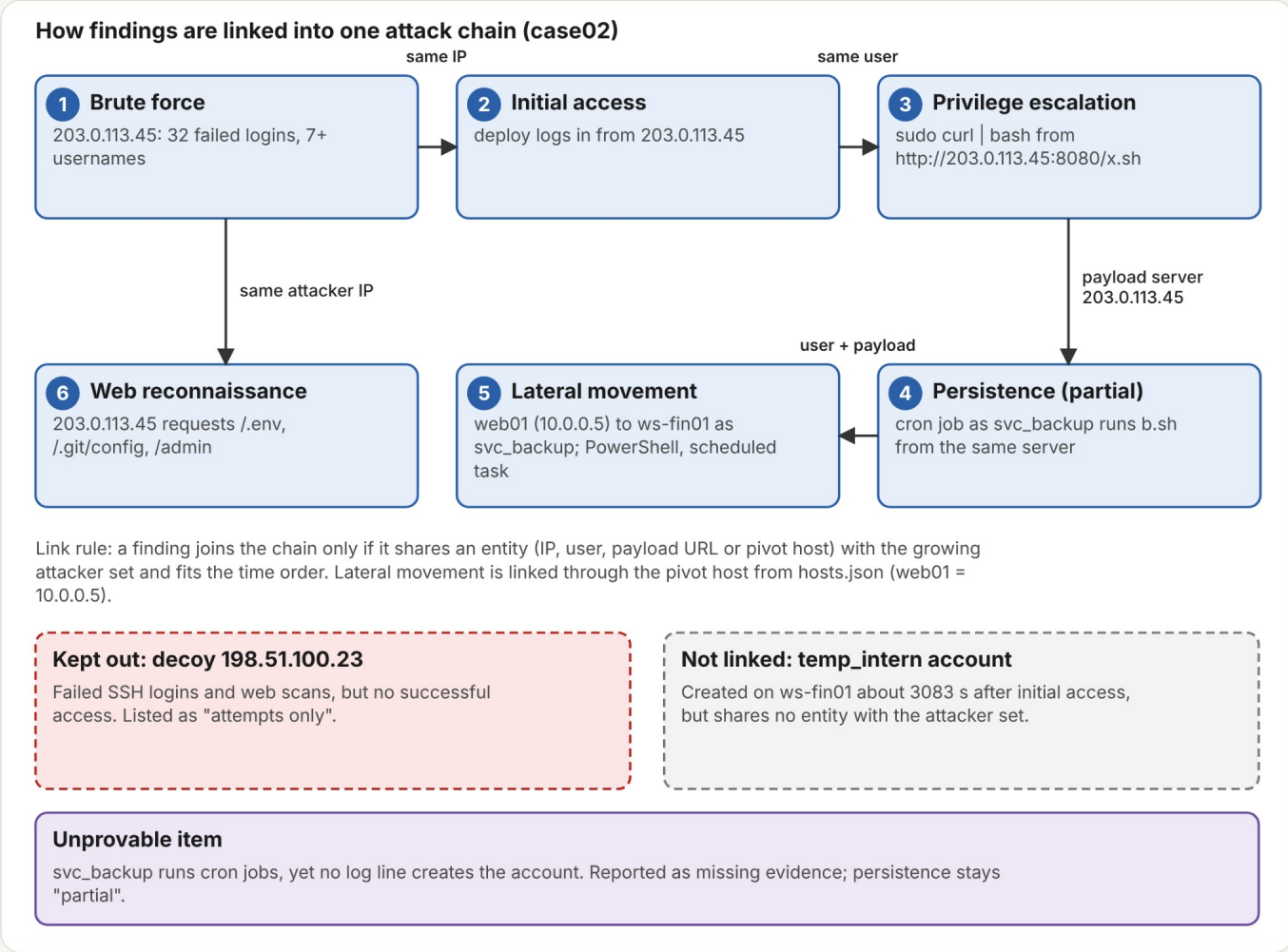
Rule	It fires when	Default threshold
Brute force	Many failed logins from one address	8 failures in 10 minutes
Initial access	A login succeeds soon after the guessing	Within 30 minutes
Privilege escalation	Admin command that downloads and runs code	Risky command patterns
Persistence	New account or scheduled job after access	Risky command patterns
Lateral movement	Hacked machine logs into a second one	Within 15 minutes
Web reconnaissance	Many page requests, mostly “not found”	15+ requests, 60% errors

No hard-coding: the attacker address, user and payload link are learned from the logs, never typed into the code.

STEP 3 · LINK THE CHAIN

Linking findings into one story

1. Start with the address that guessed passwords and got in
2. Add findings that share a clue: the same user, link or second machine
3. Keep the decoy and unrelated account out
4. Report missing evidence instead of guessing



WHAT THE SYSTEM FOUND

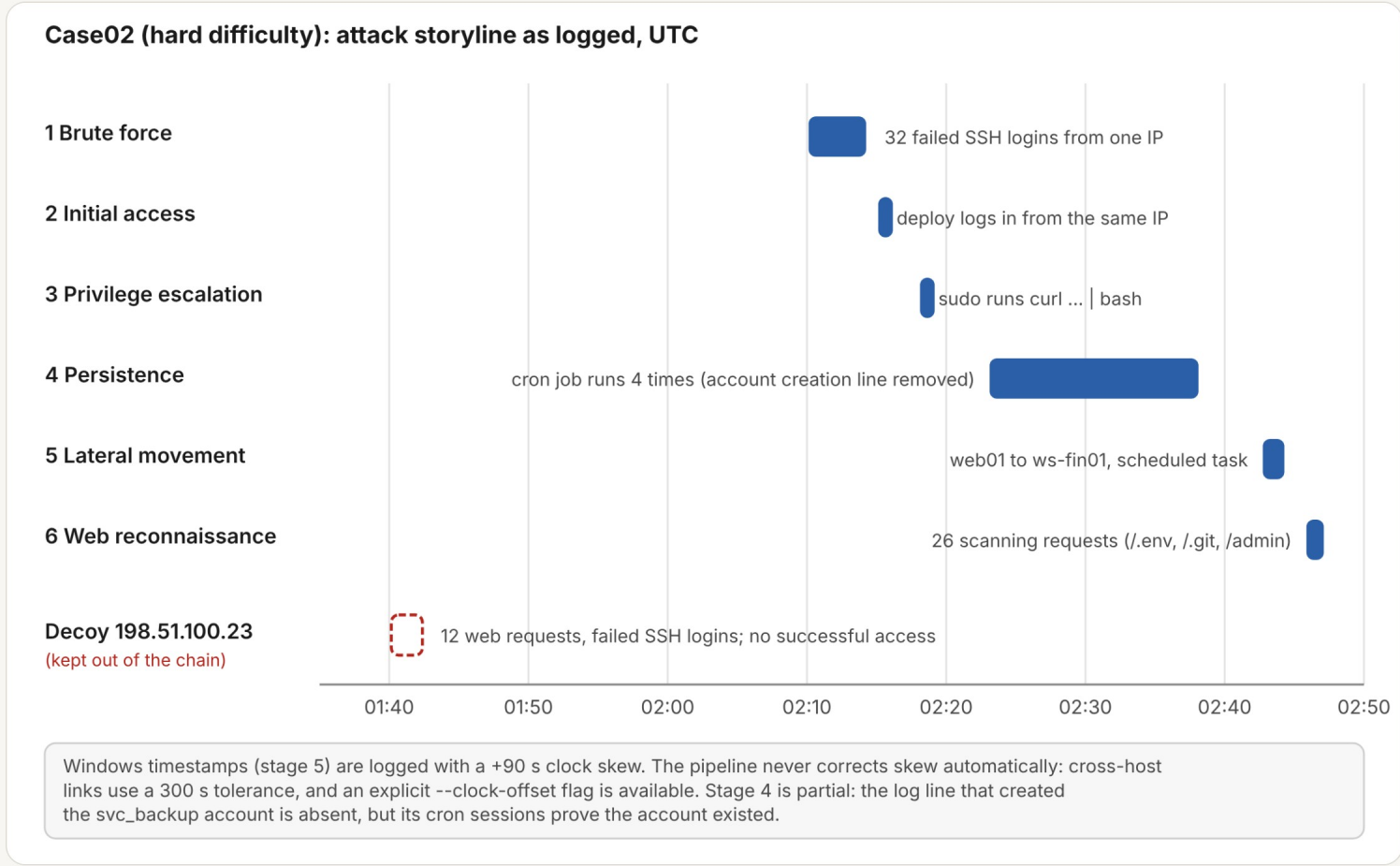
The attack on a timeline

Hard case, times in UTC

Six stages, about 37 minutes

The decoy has its own lane: it attacked but never got in

Windows events are logged 90 seconds late



STEPS 4 AND 5 · FACTS, THEN THE AI

The AI only rewrites proven facts

FACT FROM THE RULES

User deploy ran a sudo command on web01 that downloads and runs a script. **[AUTH-L330]**

→

SENTENCE FROM THE AI

The user deploy gained admin rights on web01 and ran a command that downloaded and executed a script. **[AUTH-L330]**

Small local model
qwen2.5:7b-instruct, free, runs on a laptop with Ollama

Same answer each time
Temperature 0 and a fixed seed

Strict rules
Add nothing new and cite the evidence IDs

AGENT ORCHESTRATION

Eight agents, one controller

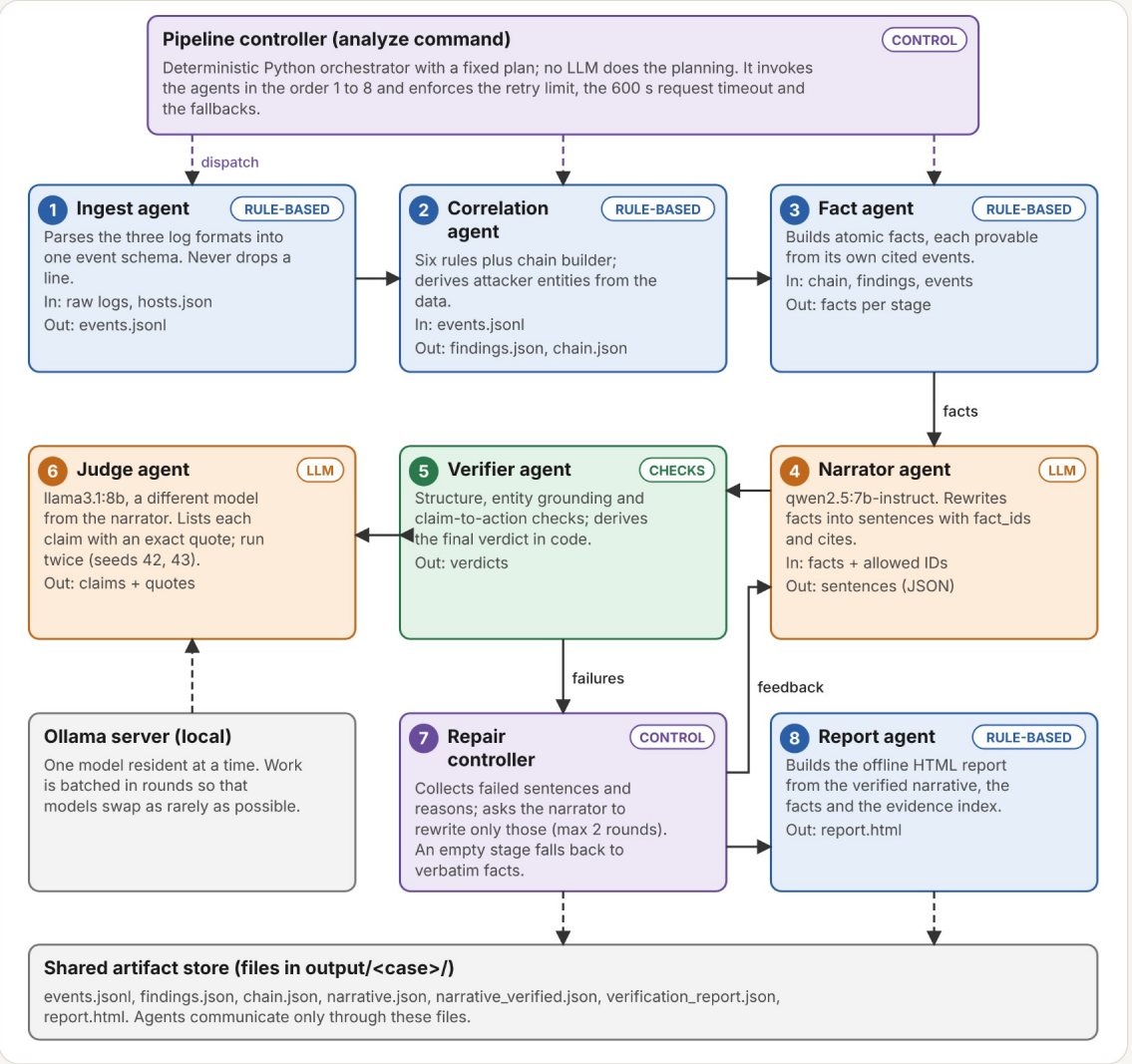
Controller: a fixed plan runs the agents in order and enforces the limits

Only two agents use an LLM: the Narrator and the Judge

Agents share files, not messages, so every hand-over can be inspected

Repair loop: failed sentences go back to the Narrator, at most two rounds

On purpose: a fixed plan, not a model choosing its own steps, so runs are repeatable



STEP 6 · CHECK EVERY SENTENCE

Four layers of checking

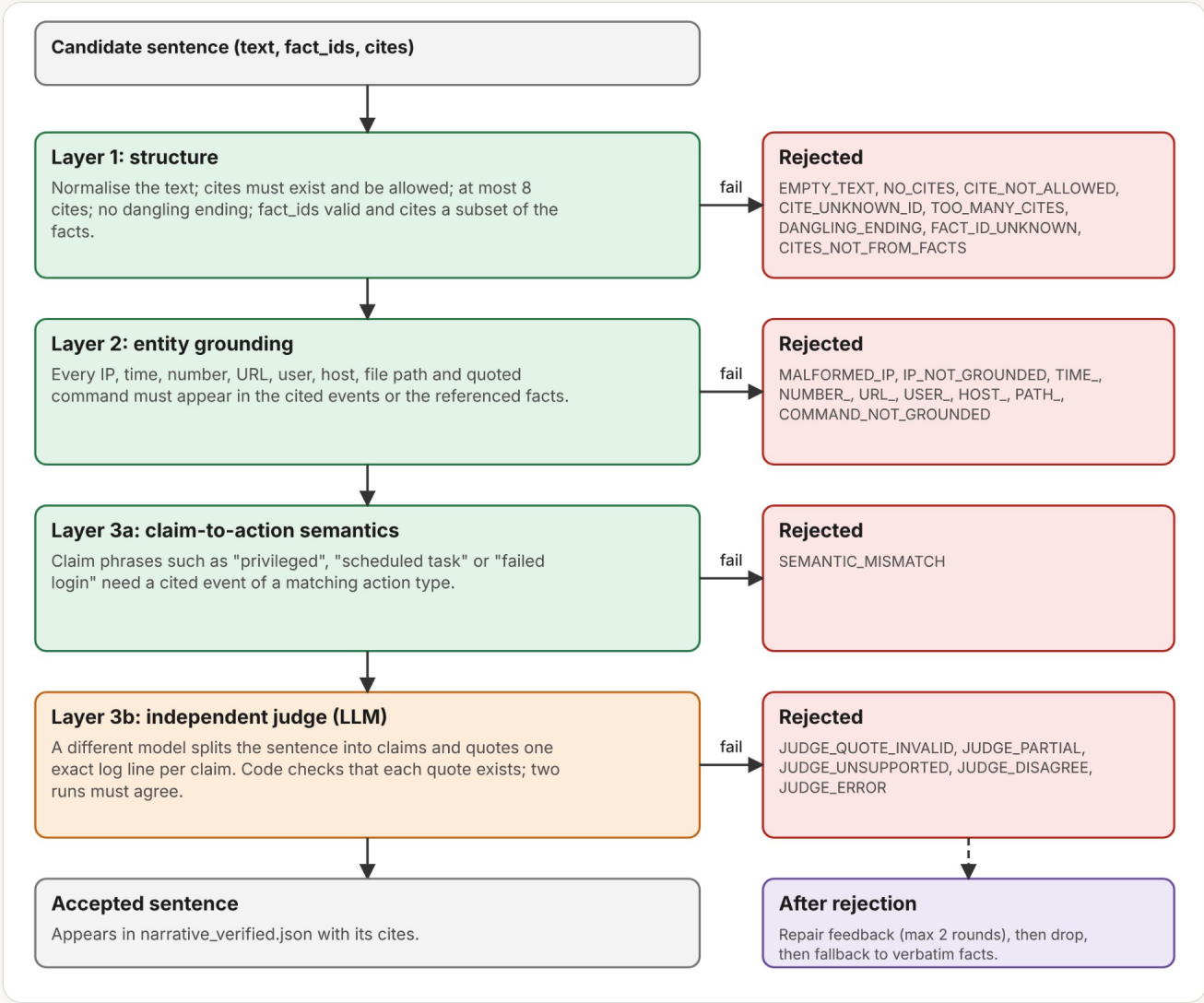
1 Structure: are the cited evidence IDs real and allowed?

2 Entities: does every IP, time and name appear in the cited lines?

3a Meaning: does the claim match the type of event cited?

3b Judge: a second AI quotes the exact log line for each claim

The final verdict is decided by code, not by the AI.



STEP 6 · REPAIR LOOP

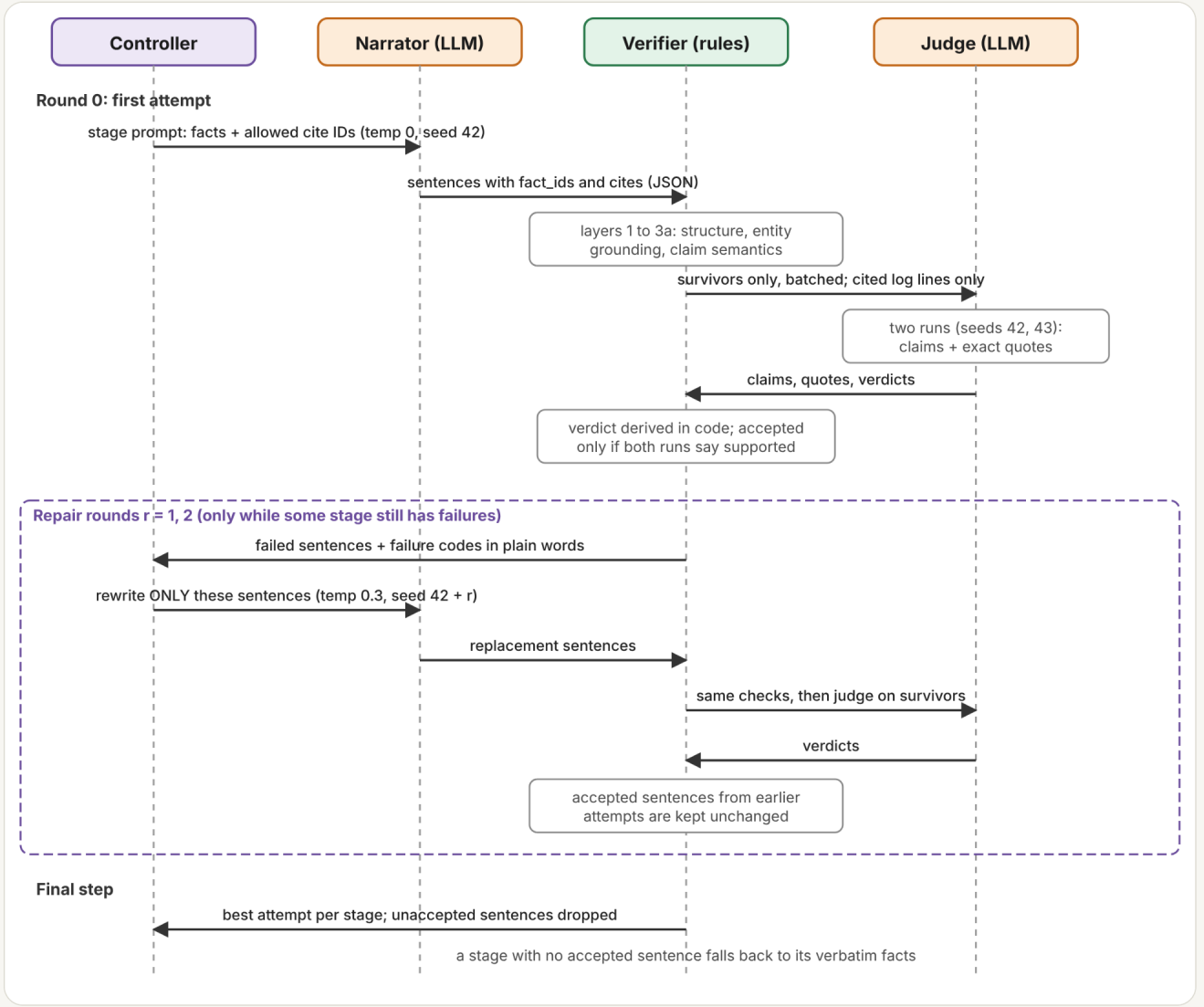
Fix what failed, keep what passed

Round 0: the Narrator writes a first draft

Checks: the Verifier and the Judge test each sentence

Rounds 1 and 2: only failed sentences go back, with the reasons

Still failing? It is dropped, or replaced by the plain verified fact



THE RESULT

The final report

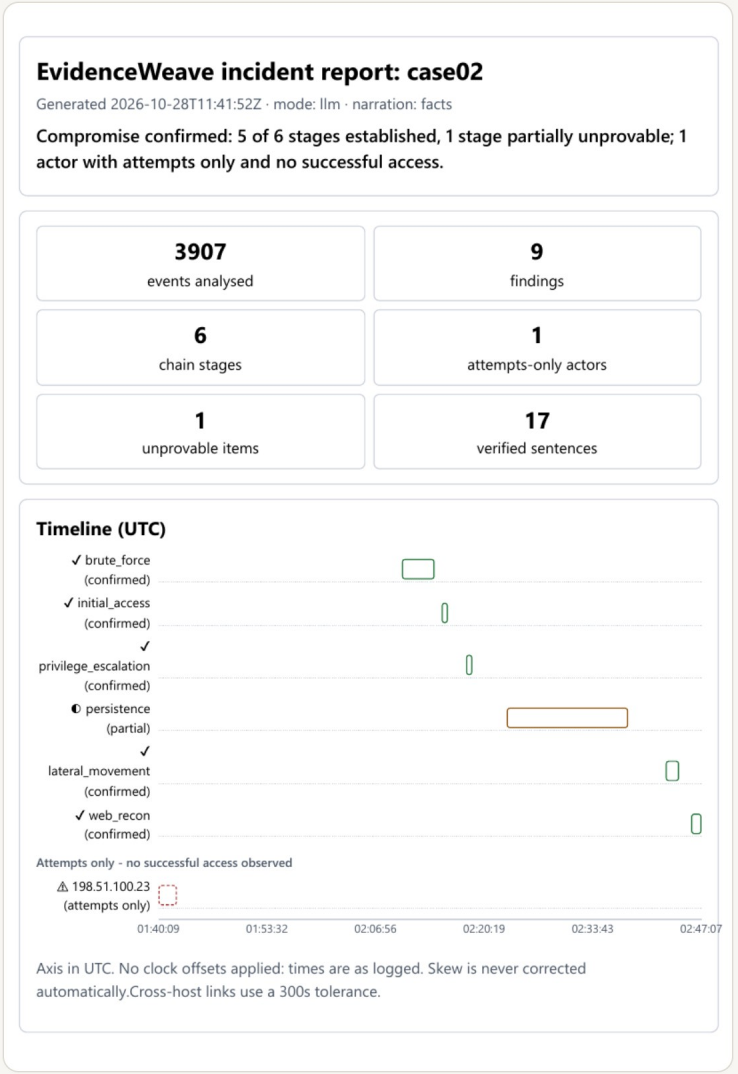
One offline file, easy to e-mail

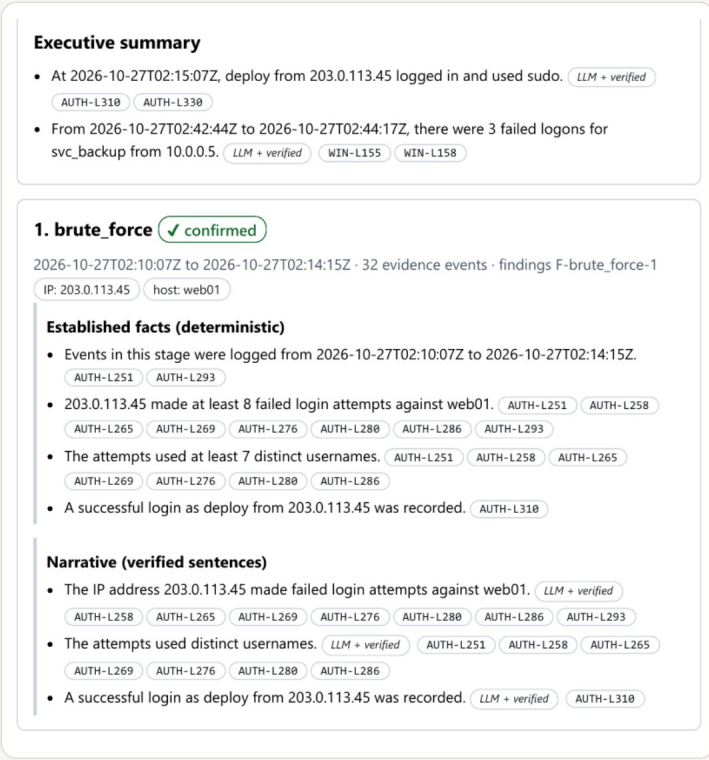
Timeline with a separate lane for the decoy

Facts and verified sentences side by side

Click an evidence ID to see the raw log line

A section for what could **not** be proven





THE PROOF · RESULTS

Three numbers to remember

329/329

Attack lines found

On four new test cases, with no false additions

91.7%

Attack steps in the story

Same AI model on raw logs without rules: only 8.3%

119/120

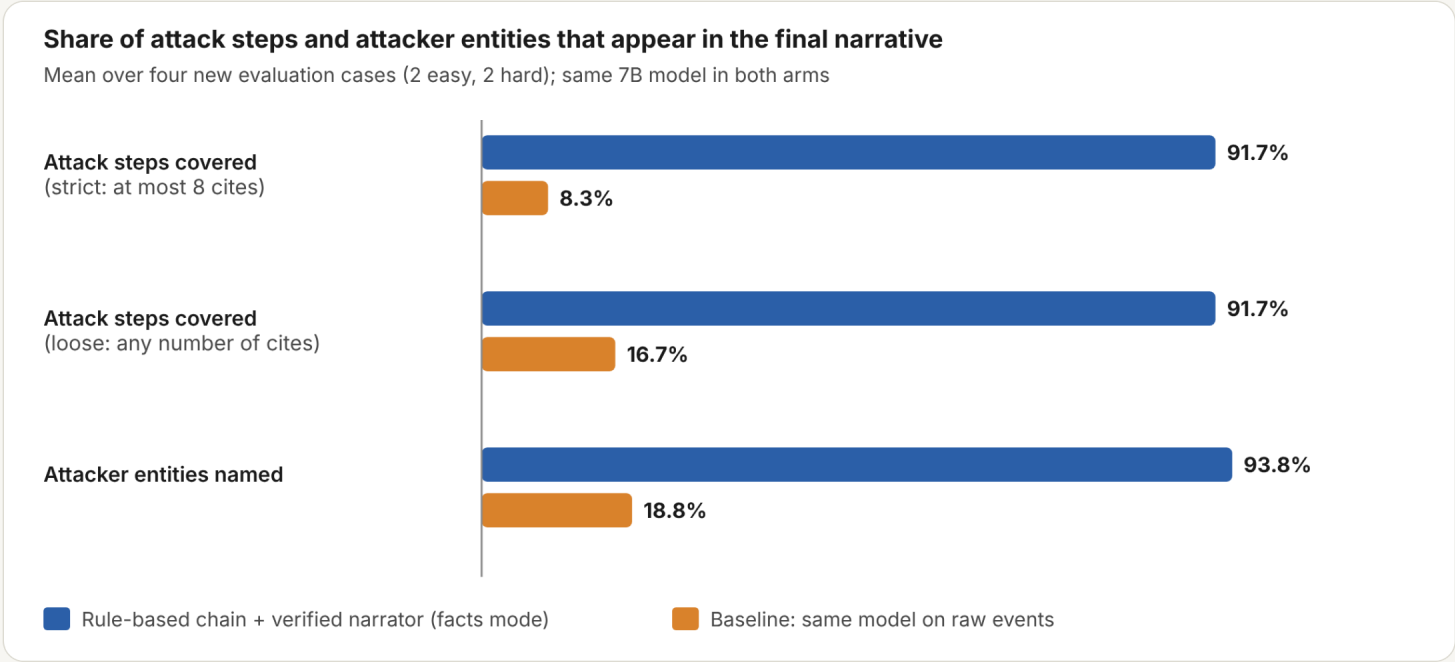
Fake sentences rejected

Deliberately corrupted sentences the checker had never seen

Test cases come from the same generator as the rules, so this shows the rules work across its variations, not on real logs yet.

THE PROOF · WITH RULES VS WITHOUT

Same AI, very different results



Blue: rules + AI + checker

Orange: the same AI on raw logs only

The raw-log version also labelled routine jobs as threats, and was about 4× slower with 10× more tokens before any checking.

HONEST LIMITS

What the results do not show

Synthetic data

No real-world logs tested yet

AI-labelled audit

The 32-sentence audit was labelled by an LLM, so it is not independent

Small samples

Four cases, one run each

One miss

1 of 120 corrupted sentences slipped through: a claim with no checkable name

False rejections

2 of 28 good sentences were wrongly rejected

Slow checking

A full run with the judge takes about 17 minutes

WHAT COMES NEXT

Future work

1. Test on real or public log collections
2. Add a human audit of the sentences
3. Check counts, absence claims and interpretations
4. Write a richer executive summary
5. Estimate clock skew when a reliable anchor event exists
6. Compare larger models

IN ONE SENTENCE

Rules find it. AI explains it. A checker proves it.

Thank you. Questions?

Nithya Maria Mathew · URK23CS7004 Demian Sharwin Xaxa · URK23CS7011